

Predicting OVER vs UNDER the Sportsbook Total Line in NBA Games

UE24CS352A Machine Learning Mini-Project · Problem 15: *The Bank is Open: AI in Sports Gambling* · Monika P M (PES1UG24CS275), More Atharv Sachin (PES1UG24CS276) · Code: [private GitHub repo - link placeholder]

1. Problem statement

Our project is a simplified, classification version of the paper *The Bank is Open: AI in Sports Gambling* (Bucquet & Sarukkai), which studies the NBA Over/Under betting indicator using recent team form, opponent information, rest and travel features. The paper regresses the raw combined point total. We instead define a binary task: **for each NBA game, predict whether the final combined score will be OVER (1) or UNDER (0) the sportsbook total line**. We do not predict the raw game total. A model is useful only if it classifies correctly more often than chance, and a bettor would need roughly 52.4% accuracy to break even at standard -110 odds (the paper cites 52-53%).

2. Dataset

Source: Kaggle, *NBA Betting Data | October 2007 to June 2026* (user cviaxmiwnptr), <https://www.kaggle.com/datasets/cviaxmiwnptr/nba-betting-data-october-2007-to-june-2024>; file `nba_2008-2026.csv`. Per the dataset page, games from Oct 2007 to Jan 2023 come from Sportsbook Reviews Online (the paper's odds source), and later games from ESPN and Sportsbook Review. It has 24,440 games (2007-10-30 to 2026-06-13), 30 teams and 27 columns including final scores, quarter scores, spread, moneylines and `total`. `total` is the **sportsbook total line** (the file does not state whether it is an opening or closing line). The actual game total is `score_home + score_away`. We drop the 279 pushes (actual total equal to the line), leaving 24,161 games; the first games of each team's history lack last-3-game statistics, so 24,112 are modelled. The classes are balanced (Over rate 49.6% train, 50.8% validation, 50.6% test).

3. Approach

Chronological split (no shuffling): train 2008-2022 (18,849 games), validation 2023 (1,311), test 2024-2026 (3,952).

Hyper-parameters were chosen on validation ROC-AUC from small grids; the test set was used only for the final evaluation, and final models are trained on the training seasons only.

Features (38). All team-performance features are computed only from games played **before** the game being predicted (statistics are shifted by one game before any rolling or season-to-date average); the current game's score is used only to create the label. For both the home and away team: (i) last-3-game averages of points scored, points allowed, total points, margin and win rate; (ii) the last-3 average of the opponents' season-to-date points scored/allowed, as an opponent-strength measure in the spirit of the paper's opponent season averages; (iii) the team's season-to-date points scored, allowed and total; (iv) rest days, a back-to-back flag and games in the last 7 days; (v) travel distance in km from the previous game's city. Game-level features: `line_total`, absolute spread, home-favoured flag, playoff flag, expected totals from last-3 and season-to-date form, each expected total minus the line, both-teams-back-to-back and travel difference. **Note: `line_total` (the sportsbook total line) is used as an input feature.** The travel feature uses **approximate city-level coordinates for each team's arena city and the Haversine distance**; the coordinate table (`src/arenas.py`) was written from general geographic knowledge, not from a cited dataset, and precision is about 1 km. Missing values (at most 3.3% per feature) are imputed with the training-set median, and features are standardised for Logistic Regression and the neural network.

Models. Logistic Regression; **Random Forest (baseline model)**; XGBoost; Neural Network (scikit-learn MLP). Selected settings: LR $C=1.0$; RF 300 trees, max depth 4, min leaf 10; XGBoost 100 trees, depth 4, learning rate 0.03; MLP hidden layers (32,), L2 alpha 0.01, early stopping. Two reference baselines need no learning: always predicting the training-set majority class (Under) and a seeded coin flip.

4. Implementation overview

The code is in a Python project: `src/features.py` (long team-game table, rolling/season statistics, rest, travel, label), `src/arenas.py` (coordinates) and `src/train.py` (preprocessing pipelines, tuning, evaluation, plots). Libraries: pandas, NumPy, scikit-learn, XGBoost, Matplotlib. `python src/train.py` reproduces every number below (random seed 42). As a leakage check we recomputed the last-3-game points of 5 random games by hand and they matched the feature values.

5. Results (test set, 2024-2026, n = 3,952)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	49.65%	0.502	0.476	0.489	0.494
Random Forest (baseline)	50.76%	0.527	0.255	0.344	0.518
XGBoost	50.00%	0.507	0.429	0.464	0.509
Neural Network (MLP)	50.53%	0.515	0.380	0.437	0.505
Majority-class baseline	49.44%	0.000	0.000	0.000	n/a
Coin-flip baseline	49.67%	0.502	0.497	0.500	n/a

Precision, recall and F1 treat OVER as the positive class; the decision threshold is 0.5. The 95% confidence half-width of an accuracy near 50% on 3,952 games is about ± 1.6 percentage points. The break-even accuracy is about 52.4%, so no model reaches it.

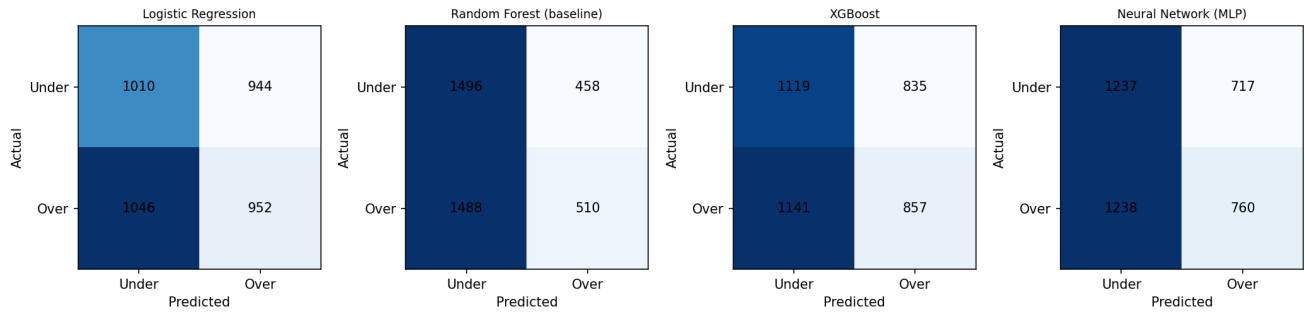


Figure 1. Test-set confusion matrices (rows: actual, columns: predicted).

6. Conclusions

- Random Forest had the highest accuracy we observed (50.76%) and the highest ROC-AUC (0.518), but this is within the estimated uncertainty (± 1.6 points) of the majority-class (49.44%) and coin-flip (49.67%) baselines. It also has low recall (0.26) because it predicts Under more often.
- All four models land between 49.65% and 50.76%. **None of the models demonstrates a meaningful predictive advantage** over chance, and none approaches the $\sim 52.4\%$ break-even level. We therefore make no claim of beating the sportsbook.
- This is consistent in spirit with the paper's own modest 51.5% result and suggests the sportsbook total line already absorbs most of the information in simple recent-form, rest and travel features. Random Forest importances are spread thinly across many features (largest: `l3_total_minus_line` 0.061), so we do not interpret any single feature.

Limitations / future work. Only team-level features were used (no injuries or player data); validation (1,311 games) is small, so hyper-parameter choices are noisy; travel uses approximate city coordinates; the line's opening/closing status is unknown; results are for one chronological split. Possible extensions: player-level and injury data, line-movement features, longer game histories (as in the paper's LSTM direction), and probability calibration.

References

[1] A. Bucquet and V. Sarukkai, *The Bank is Open: AI in Sports Gambling* (course report and poster provided with the assignment). [2] *NBA Betting Data | October 2007 to June 2026*, Kaggle dataset by cviwxmiwnptr (see URL above). [3] Sportsbook Reviews Online, Historical NBA Scores and Odds Archives (original odds source named in [1] and in the Kaggle dataset description). [4] scikit-learn and XGBoost libraries.